

1.1) neuron computer $S(z) = \psi(z \cdot \beta_{OR} + b_{OR})$

ψ : if $z \geq 0$ return 1 (or $\mathbb{1}_{(z \geq 0)}$)
else return 0

β_{OR} is an array of 1s: $[1, 1, \dots, 1]$

b_{OR} is -1

$$z[0, 0]: (0 \cdot 1 + 0 \cdot 1) - 1 = -1, \psi(-1) = 0$$

$$[1, 0]: (1 \cdot 1 + 0 \cdot 1) - 1 = 0, \psi(0) = 1$$

$$[1, 1]: (1 \cdot 1 + 1 \cdot 1) - 1 = 1, \psi(1) = 1$$

1.2) masked logical OR

$$z \rightarrow g(z; c) := \psi(z \cdot \beta_{MOR} + b_{MOR})$$

β_{MOR} and b_{MOR} depend on c ? we can use same ψ
need c in equation,

$\beta_{MOR} = c$, $b_{MOR} = -1$, same ψ
dot product

$$c[1, 0]$$

$$z[0, 0]: (0 \cdot 1 + 0 \cdot 0) - 1 = -1, \psi(-1) = 0$$

$$\downarrow [1, 0]: (1 \cdot 1 + 0 \cdot 0) - 1 = 0, \psi(0) = 1$$

$$\downarrow [1, 1]: (1 \cdot 1 + 1 \cdot 0) - 1 = 0, \psi(0) = 1$$

$$\downarrow [0, 1]: (0 \cdot 1 + 1 \cdot 0) - 1 = -1, \psi(-1) = 0$$

1.3) I don't see how the same $\psi(z)$ can be used for the perfect match too

$$z \rightarrow h(z; c) := \psi(z \cdot \beta_{MA} + b_{MA}) = \begin{cases} 1 & z = c \\ 0 & \text{otherwise} \end{cases}$$

dot product with inverse to find the solution

$$z \cdot c^{-1} + z^{-1} \cdot c = 0$$

$$z(1-c) + c(1-z)$$

$$z - zc + c - cz$$

$$z + c - 2zc$$

$$z + c(1-2z)$$

$$c + z(1-2c) \text{ reformat for } z \cdot \beta_{MA} + b_{MA}$$

$$\downarrow$$

$$z(1-2c) + c$$

$$\beta_{MA} = 1 - 2c, b_{MA} = \|c\|$$

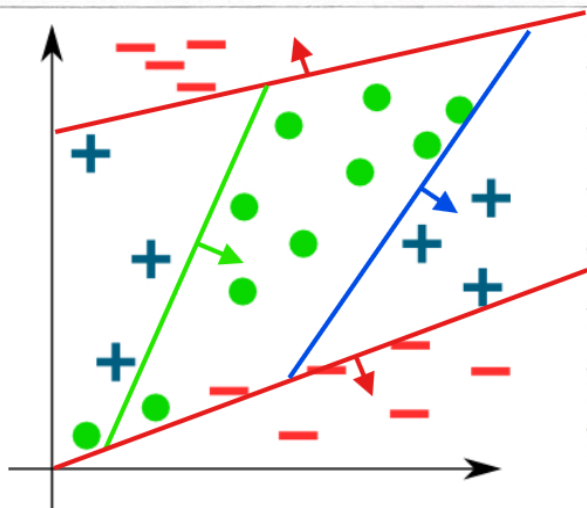
$\psi(z)$: if $z = 0$ return 1
else return 0

$$c [1, 0] \quad \|c\| = 1$$

$$z [1, 0]: (1 \cdot -1 + 0 \cdot 1) + 1 = 0, \psi(0) = 1$$

$$\downarrow [0, 1]: (0 \cdot -1 + 1 \cdot 1) + 1 = 2, \psi(2) = 0$$

$$\downarrow [1, 1]: (1 \cdot -1 + 1 \cdot 1) + 1 = 1, \psi(1) = 0$$



1.4) Three-layer Network

Hypercube corners:

Red top right: $(0, 0, 0)$
 Red bottom left: $(0, 0, 0)$

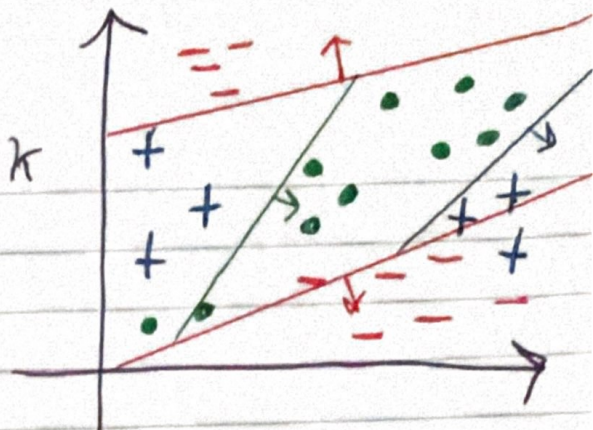
Red top left: $(0, 0, 0)$

Red bot right: $(1, 1, 0)$

Blue top right: $(1, 0, 0)$

Blue bot left: $(0, 1, 0)$

Green arbitrary: $(1, 1, 1)$



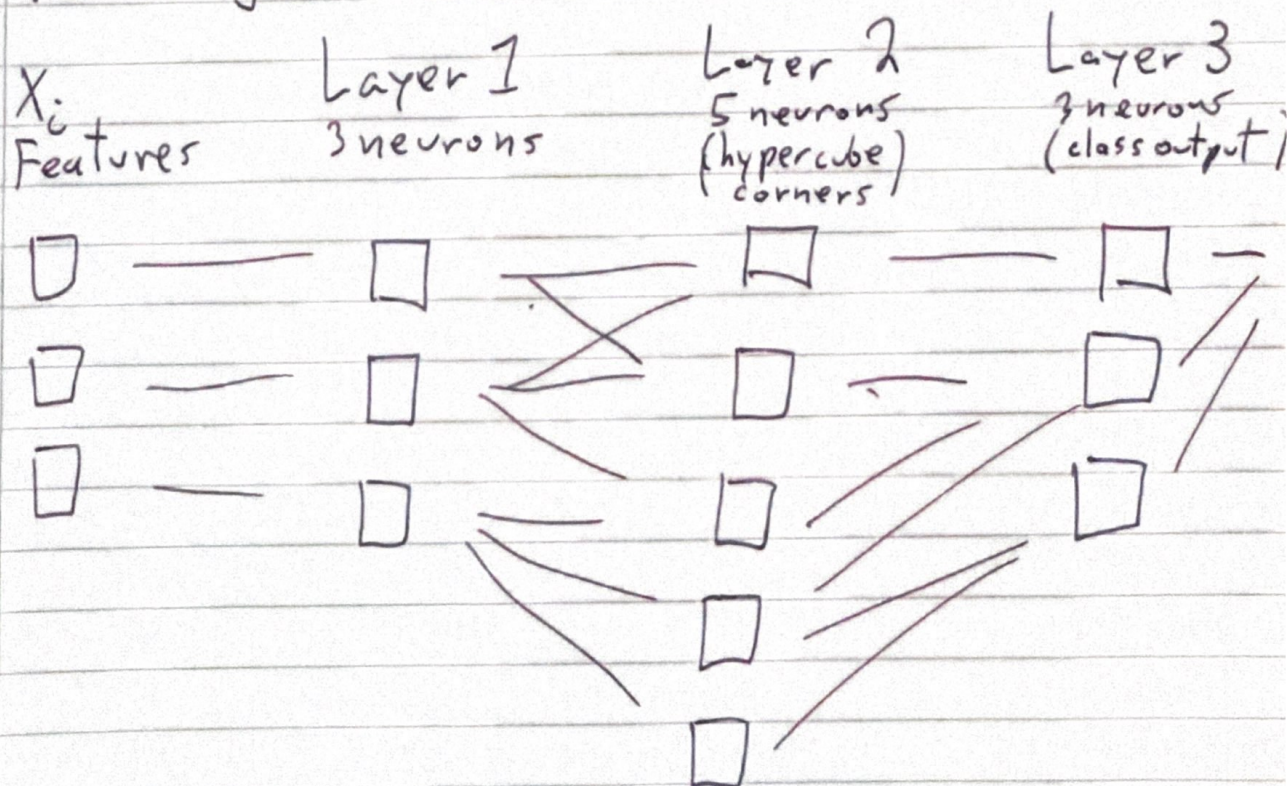
Red: , 

Blue: , 

Green: 

The equations used in the networks 2nd and 3rd layers to produce is the perfect match equation we made earlier $\psi(z \cdot B_{MA} + b_{MA}) = z \cdot (1 - 2c) + ||c||$.

Resulting Network:



this could be generalized to arbitrary many ~~data~~ input dimensions and arbitrary label distributions by adding more classifications and utilizing a higher dimensional hypercube to pick more corners and then adding more one-hot encodings and more neurons. It can scale up that way.

The potential problems with this zero training loss classifier is that the boundaries we draw are hard to capture more complicated data, and the amount we ~~would~~ need to scale with even more data would be unmanageable and will run into overfitting the training data.

$$\forall i \in \llbracket 1, n \rrbracket: \phi_i = \text{id} \downarrow$$

$$\exists. Zg.: \quad \phi_n(B_n(\phi_{n-1}(B_{n-1}(\dots) + b_{n-1}) + b_n)) = A_n x + a_n$$

Proof by induction:

$$n=1 : \quad \phi_1(B_1 x + b_1) \stackrel{!}{=} A_1 x + a_1$$

$$\stackrel{\phi_1 = \text{id}}{\Rightarrow} B_1 x + b_1 = A_1 x + a_1 \quad \checkmark$$

IA:

$$\forall i \in \llbracket 1, n \rrbracket: \phi_i = \text{id} \downarrow$$

$$\phi_n(B_n(\phi_{n-1}(B_{n-1}(\dots) + b_{n-1}) + b_n)) = A_n x + a_n$$

$n \Rightarrow n+1$

$$\phi_{n+1}(B_{n+1}(\phi_n(B_n(\phi_{n-1}(B_{n-1}(\dots) + b_{n-1}) + b_n) + b_{n+1}))$$

$$\stackrel{\phi_{n+1} = \text{id}}{\Rightarrow} \phi_{n+1}(B_{n+1}(A_n x + a_n) + b_{n+1}) \stackrel{\phi_{n+1} = \text{id} \downarrow}{=} \underbrace{B_{n+1} A_n x}_{= A_{n+1}} + \underbrace{B_{n+1} a_n + b_{n+1}}_{= a_{n+1}}$$

$$= A_{n+1} x + a_{n+1} \quad \checkmark$$